Skill的扩展能力

- 来源: https://class.imooc.com/lesson/5287#mid=105139
- 阶段: 阶段五: 【能力封装】可复用技能
- 周: 第10周 Skill：可复用技能封装
- 模块: Skill核心概念与扩展能力
- 课节: 图文 1-2 Skill的扩展能力

1-2 Skill的扩展能力

Skill的扩展能力

> [!TIP] 阅读指南 前面几节我们接触的 Skill——从 Humanizer 到 Algorithmic Art——本质上都是在做同一件事：用文本指导大模型如何完成任务。Humanizer 教大模型识别并去除 AI 痕迹，Algorithmic Art 教大模型生成 p5.js 代码。这些 Skill 的 SKILL.md 里写的是规则、规范、步骤，大模型读完自己就能执行。 但有一些任务是文本指导搞不定的。比如：转换一个文件格式、压缩一张图片、解析一份 PDF、调用一个 API。这些事情需要 Agent 真正去"动手"——运行一段代码、执行一个脚本、调用一个函数。 这就是本节要讨论的主题：Skill 如何与工具协作，让系统不仅能"想"，还能"做"。

文本指引的边界

先搞清楚一个问题：什么情况下只用文本就够了？

回顾 Humanizer 的工作方式：大模型读取 SKILL.md 中的规则（“检测否定式排比” “去掉教科书式论述腔调”），然后在生成回答时应用这些规则。整个过程停留在文本生成层面——大模型读规则，写文本，结束。

类似地，代码审查 Skill 也是文本指引：SKILL.md 里写了命名规范、格式化标准、错误处理要求，大模型读完之后去审查用户提供的代码，给出评审意见。

这些场景有一个共同点：大模型本身具备完成该任务所需的所有能力，缺的只是专业知识。Skill 补充了知识缺口，大模型就能自己干。

但换个场景就不行了。假设你要创建一个 Skill，功能是"把 Markdown 文件批量转换成 PDF"。大模型再怎么聪明，也没法凭空在服务器上调用 pandoc 或 wkhtmltopdf（生成PDF的库）——它需要

Agent 帮它执行这些操作。

这就是文本指引的边界。跨越这个边界，就需要引入工具（工具）。

工具（工具）

Skill 如何调用工具

在讨论具体细节之前，需要先理清一个三方关系：Skill、大模型、Agent。这三者各司其职：

关于 Agent，本书第 11 章会有专门的深入讨论。这里你可以暂时把它理解为运行 Skill 的那个 AI 工具——比如 Claude Code 或 Qoder。回忆一下，前面几节我们实际使用 Skill 时，都是在这些工具里操作的——Skill 必须寄生在 AI 工具中才能运转。这个宿主，就是 Agent。

有了这个前提，来看三者的分工：

```
Skill（知识）
  │
  │ 提供指令和规则
  ▼
大模型（大脑）
  │
  │ 读取 Skill 后，判断该调用哪个脚本、传什么参数
  ▼
Agent（手脚）
  │
  │ 实际执行：运行 Python 脚本、调用系统终端、操作文件
  ▼
最终结果返回给用户
```

Skill 只负责"教"，不负责"做"。真正动手的是 Agent——它才是那个运行脚本、执行命令的角色。大模型在中间扮演决策者：它读取 Skill 里的指令，理解用户当前的需求，然后告诉 Agent 该干什么。

打一个比方：Skill 是一本方子书，大模型是读方子的老中医，Agent 是抓药熬药的学徒。缺了任何一个环节，药都熬不出来。

在 Skill 的目录结构中，scripts/ 目录就是专门为这个场景设计的：

```
scripts/
```

```
my-skill/
├── SKILL.md              ← 核心指令（必需）
├── scripts/              ← 可执行脚本（可选，但这里就是重点）
│   ├── convert.py        ← Python 脚本
│   ├── generate.sh       ← Shell 脚本
│   └── template.js       ← Node.js 脚本
├── references/           ← 参考文档（可选）
└── assets/               ← 输出资源（可选）
```

与 SKILL.md 的"教大模型怎么做"不同，scripts/ 里的文件是交给 Agent 去执行的。大模型读取 Skill 触发后，看到 SKILL.md 中指向 scripts/convert.py 的指令，就会通知 Agent 去运行这个脚本。Agent 通过第 6 章讨论的 Function Calling 机制来完成调用。

一个直观的例子

假设我们写一个"PDF 内容提取" Skill：

```
pdf-content-extractor/
├── SKILL.md
└── scripts/
    ├── extract_text.py     ← 提取 PDF 中的文字
    └── split_pages.py      ← 按页切分内容
```

SKILL.md 中会这样引导大模型：

```
当用户需要提取 PDF 里的文字内容时：

1. 运行 scripts/extract_text.py <pdf文件路径>
   这个脚本会返回 PDF 里的文字内容

2. 根据返回结果回答用户的问题
```

大模型看到这些指令后，通知 Agent 调用代码执行功能来运行 extract_text.py，获取结果，然后回复用户。整个过程对用户来说是透明的——用户只知道自己问了"这份 PDF 里写了什么"，后台已经完成了 Skill 指导、模型决策、Agent 执行三个步骤的接力。

script 与执行者的对应关系

Agent 执行这些脚本时，依赖的是第 4 章讲过的 Function Calling 机制。不同类型的脚本对应不同的执行方式：

脚本类型	执行方式	典型场景
Python 脚本	通过 Python 解释器运行	数据处理、格式转换、API 调用
Shell 脚本	通过系统终端运行	文件操作、批处理、系统命令
Node.js 脚本	通过 Node 运行时运行	前端构建、npm 包调用

Skill 的 scripts/ 目录把"该用什么工具"和"怎么用"打包在一起——大模型不需要自己想办法安装依赖、拼接命令行，告诉 Agent 照着执行就行。

References：为大模型补充背景知识

references/ 目录装的是大模型执行任务时可能需要查阅的参考资料，而不是直接执行的代码。

它的使用场景很明确：当 SKILL.md 的核心指令不足以覆盖所有细节时，把详细内容拆到 references/ 中，保持 SKILL.md 精简。

```
my-skill/
├── SKILL.md                ← 核心流程 + 导航
└── references/
    ├── finance.md          ← 财务相关表结构
    ├── sales.md            ← 销售相关查询模式
    └── api_docs.md         ← API 文档
```

与 scripts/ 的区别在于：script 是执行的，reference 是阅读的。Agent 不会自动运行 references/ 下的文件，只有当大模型（通过 SKILL.md 的指引）判断"去查阅 references/finance.md 了解表结构"时，才会去读取。

这种设计解决了 SKILL.md 膨胀的问题。如果一个 Skill 涉及多个业务领域（比如财务、销售、市场各有不同的数据模式），全部塞进 SKILL.md 会让文件变得臃肿。分拆到 references/ 后，大模型只加载当前任务需要的部分，节省上下文窗口。

这其实就是 Skill 的渐进加载能力：SKILL.md 是入口，永远加载；references/ 是按需查阅的词典，只在任务涉及某个领域时才被翻开。用户问财务问题时，大模型只加载 finance.md，sales.md 和 api_docs.md 不会被拖进上下文——既节省了资源，也让大模型能专注于当前问题的领域知识。

Assets：准备输出需要的素材

assets/ 目录存放的是 Agent 在生成最终输出时需要用到的素材文件。这些文件不会被加载到大模型的上下文中，而是直接作为输出的一部分被使用。

典型场景：

- 模板文件——assets/template.pptx，Agent 把它作为基准文件，复制并填充内容后输出
- 图片和图标——assets/logo.png，Agent 在生成 HTML 页面时引用
- 前端脚手架——assets/hello-world/，Agent 复制整个目录作为项目的初始代码

什么时候该用脚本，什么时候该用文本

这里给出一个简单的判断原则：

- 如果任务可以用规则描述清楚，大模型自己就能指导 Agent 完成 → 写在 SKILL.md 正文中（文本指引）
- 如果任务需要执行确定性的、反复出现的代码逻辑 → 写成 scripts/ 下的脚本，交给 Agent
- 如果任务需要大模型查阅大量背景信息 → 放到 references/ 中
- 如果任务需要输出固定的模板或资源文件 → 放到 assets/ 中

以 Humanizer 为例：它只用了 SKILL.md 正文，没有 scripts、没有 references、没有 assets。因为它的任务就是"识别 AI 痕迹并改写"，这完全属于大模型自身的文本能力范畴——一台顶级跑车，加好油（知识）就能跑，不需要额外挂载设备。

但 PDF 处理 Skill 就不一样了：大模型没法直接操作出一个 PDF 文件（记住，大模型只能做决策，他永远不能直接调用工具），必须让 Agent 实际运行 Python 脚本来操作 PDF。

实战对照：Humanizer 与 PDF 处理

来看两个 Skill 的结构差异：

```
humanizer-zh-1.0.0/              pdf-processor/
├── SKILL.md                      ├── SKILL.md
                                  ├── scripts/
                                  │   ├── extract_text.py
                                  │   ├── merge_pages.py
                                  │   └── compress.py
                                  └── references/
                                      └── pdf_spec.md
```

Humanizer 只需要一个 SKILL.md，因为它的全部逻辑就是"告诉大模型哪些是 AI 痕迹"。大模型知道这些规则后，自己能完成改写。

PDF 处理 Skill 则必须有脚本——大模型无法通过文本描述来压缩一个 PDF 文件，必须让 Agent 调用 scripts/compress.py 来实际执行操作。

这个对比帮助理解 Skill 的一个设计原则：能用文本说清楚的事，就不要写脚本。脚本是对大模型能力边界的补充，而不是替代。好的 Skill 会先问自己：大模型本身能不能做这件事？如果不能，才用脚本补上。

■ 学点英语

中文	English	音标	说明
脚本目录	Scripts	/ˈskrɪpts/	Skill目录中的可执行脚本，交由Agent运行，补充大模型无法直接操作的能力
参考资料	References	/ˈrefrənsɪz/	Skill目录中的阅读型文件，大模型按需查阅，用于补充背景知识
资源目录	Assets	/ˈæsets/	存放Agent生成输出时需用的素材文件和模板
函数调用	Function Calling	/ˈfʌŋkʃn ˈkɔːlɪŋ/	Agent调用外部工具/脚本的核心机制，Agent通过它执行scripts/中的代码

下一节预告

掌握了 Skill 与工具的协作方式后，下一节我们换一个角度：与其手写繁杂的 SKILL.md，不如让 AI 来帮你写。我们将用 Qoder 的内置功能，只需描述需求，AI 就能自动生成一个规范的 Skill。